

## 분산 AI 에이전트 크루 오케스트레이션 — 기술 백서

Distributed AI Agent Crew Orchestration — Technical Whitepaper

한 두뇌, 여러 손 · 두뇌는 분산, 결정은 사람 *One brain, many hands · distributed brains, human decisions*

v1.0 · 2026 · DureClaw · [github.com/DureClaw](https://github.com/DureClaw)

### 초록 / Abstract

대부분의 공장·조직은 이미 여러 플랫폼의 인프라(엣지 디바이스, GPU 서버, Windows·macOS PC, 산업용 게이트웨이)를 잘 돌리고 있다. 문제는 그 위에 AI를 얹는 비용이다 — 모든 노드에 API 키를 심고, 데이터를 클라우드로 빼고, 통합을 새로 짜는 일은 무겁고 위험하다.

*Most organizations already run **multi-platform infrastructure** that works. The hard part is the cost of putting AI on top of it — keying every node, exfiltrating data, and rewriting integrations is heavy and risky.*

DureClaw는 갈아엎지 않는다. 이미 도는 인프라 위에 하나의 실시간 협력 버스를 얹고, 흩어진 머신을 키 없이(keyless) 한 줄로 합류시킨다. 추론은 마스터 브레인(Claude)이 흡수하고, 각 노드는 서로 다른 손(물리 액추에이터·브라우저·데스크톱 GUI)이 된다. 사람이 승인한 판단은 결정론적 룰로 컴파일되어 다음부터는 LLM 0회로 재사용된다. 데이터 수집·분석에서 시작해 자동화와 AX(AI Transformation)로 가는 시작점이다.

*DureClaw adds **one real-time collaboration bus** on top of infrastructure that already runs. Scattered machines join in one line, **keyless**. A **master brain** (Claude) absorbs inference; each node is a different kind of hands. Human-approved decisions are **compiled into deterministic rules** and replayed with **zero LLM calls**. It is the starting point — from data collection to automation to AX.*

### 1. 문제 / The Problem

흩어진 인프라에 AI를 도입할 때 반복되는 네 가지 벽:

| 벽 / Barrier               | 설명                                                                                    |
|---------------------------|---------------------------------------------------------------------------------------|
| 키·비용 / Keys & cost        | 노드마다 모델 키를 심으면 키 관리·비용·레이트리밋이 폭증한다.<br>Per-node keys explode cost and secret-sprawl.  |
| 데이터 주권 / Data sovereignty | 원시 telemetry·영상·소음을 클라우드로 빼는 순간 규제·보안 리스크. Raw data leaving the floor is a liability. |
| 통합 / Integration          | 이종 OS·CPU·도구(Pi·GPU·Win·Mac·PLC)를 잇는 일이 매번 일회성 통합 프로젝트가 된다.                           |
| 신뢰 / Trust                | LLM이 직접 실행하면 비결정성·감사 불가. Direct LLM action is non-deterministic and unauditale.       |

핵심은 *데이터 확보 우선*이다 — 수집하지 못한 데이터는 영원히 **없는다**. 완벽한 파이프라인보다, 지금 도는 인프라에서 **가장 쉬운 방법으로 데이터를 모으기** 시작하는 것이 먼저다.

*The principle is **data-acquisition-first**: data you don't capture is lost forever. Start collecting on the infrastructure you already have, the easiest way possible.*

## 2. 핵심 아이디어 / Core Idea

### 2.1 한 두뇌, 여러 손 / One brain, many hands

마스터(Claude)가 **두뇌**, 각 노드는 **서로 다른 손**이다. 같은 버스·같은 keyless 위임으로, 하나의 추론이 **물리 세계·브라우저·데스크톱** 어디로든 손을 뻗는다.

마스터 두뇌 (Claude·비전·추론) —fan-out—┌┐┐  
└─ edgeclaw — 물리 세계의 손 (shell·sensor·GPIO·LED·부저·릴레이·신호탑)  
└─ webclaw — 브라우저의 손 (fetch·DOM, CORS-free)  
└─ deskclaw — 데스크톱 GUI의 손 (스크린샷·클릭·타이핑·앱실행)  
└─ adapters — 기존 도구의 손 (pico·nano·zero·null)

### 2.2 Keyless 엣지 / Keyless edges

엣지 노드는 **모델 키를 갖지 않는다**. 자연어 작업이 오면 노드는 프롬프트를 마스터 브레인의 `/brain/exec` 로 위임하고 결과만 받는다. 비용·키·레이트리밋은 마스터 한 곳이 흡수한다.

*Edge nodes hold **no model key**. They delegate prompts to the master brain and receive results; the master absorbs all cost and secret management.*

### 2.3 결정 동결 / Decision freezing (LLM as compiler)

LLM을 매번 부르지 않는다. 사람이 **한 번 승인한 판단**은 결정론적 룰/매크로로 **동결(freeze)**되어, 같은 문제는  $\mu s$  단위로 **LLM 0회** 재생된다. 이것이 닫힌 학습 루프의 핵심이다.

*The LLM is a compiler, not a runtime dependency. An approved decision is frozen into a deterministic rule/macro and replayed with zero LLM calls.*

### 3. 아키텍처 / Architecture

#### 3.1 협력 버스 / The collaboration bus

DureClaw 버스는 **Phoenix Channel**(Elixir) 위의 WebSocket 한 채널이다. 와이어 프로토콜은 5-튜플 JSON 프레임이다:

```
[join_ref, ref, topic, event, payload]
```

- **합류 / join**: `phx_join` → `work:<WORK_KEY>` 토픽. presence에 자기 역할·능력(capabilities)을 등록.
- **분배 / dispatch**: 마스터가 `task.assign` 을 fan-out (특정 노드 또는 `broadcast` ).
- **회신 / result**: 노드가 `task.result` 를 push (join\_ref 포함 필수).
- **상태 / state**: 승인·결정은 `task.result{approved}` 또는 `state.update` 로 전파.
- **하트비트 / heartbeat**: `phoenix / heartbeat` 로 presence 유지.

REST 보조 API: `POST /api/task` , `GET /api/task-result/:id` , `/api/presence` , `/api/work-keys/latest` , `/api/health` . 인증은 Bearer 토큰( `OA_H_SECRET` ).

#### 3.2 마스터 브레인 / The master brain

마스터는 노드들의 fan-in을 수집해 추론하고 **제안(propose)**만 한다 — 직접 실행은 0. keyless 위임 엔드포인트 ( `/brain/exec` )로 옛지의 자연어 작업을 대신 처리한다. 이로써 *비전·추론은 중앙, 행동은 옛지로* 분리된다.

#### 3.3 노드 패밀리 / The node family

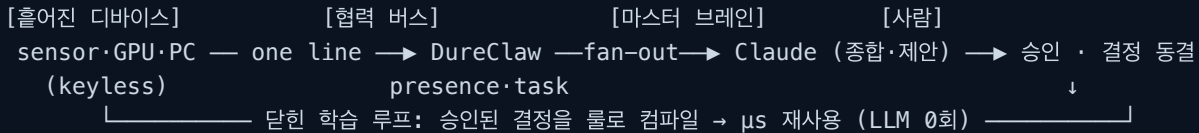
**네이티브 노드 / Native nodes** — 버스 우선 설계, 한 줄로 합류, keyless:

| 노드       | 손의 종류 / Hands                                    | 구현                                                                                                                         |
|----------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| edgeclaw | 물리 세계 — shell·sensor·GPIO·LED·부저·릴레이·신호탑·PA 음성   | 단일 정적 Go 바이너리, No-CGo.<br>Win·Mac·Linux·Pi<br>Zero(armv6)·riscv64… physical-edge는 gpiochip 노출 모든 Linux(Pi·Jetson·산업 게이트웨이) |
| webclaw  | 브라우저 — fetch·DOM (CORS-free, 상시)                 | Chrome MV3 확장, 순수 JS                                                                                                       |
| deskclaw | 데스크톱 GUI — 스크린샷·클릭·타이핑·키·앱실행 + RPA record→replay | Win/macOS, 순수 Go/No-CGo (OS 내장 도구)                                                                                         |

**어댑터 / Adapters** — 기존 오픈소스 도구에 `dureclaw/` 브리지를 더해 합류: `picoclaw`(Go) · `nanobot`(Py) · `zeroclaw`(Rust) · `nullclaw`(Zig).

*Native nodes are designed bus-first; adapters bring existing tools onto the same bus with a thin bridge. Same wire protocol, same keyless delegation.*

### 3.4 메시지 흐름 / Message flow



## 4. 결정 동결의 두 실재 / Two concrete forms of freezing

추상 원칙이 아니라 실제 두 곳에서 동작한다:

1. **제조 라인 — 스킬 캐시 / Skill cache on the line.** 불량 조사 같은 판단을 1회차에 LLM이 수행하면, 그 결과를 결정론적 룰로 캐시한다. 같은 패턴은 이후 LLM 없이 즉시 적용된다. (Open MES Korea EXT-5 연동 허브.)
2. **데스크톱 — RPA 매크로 / RPA macros on the desktop.** deskclaw에서 마스터(비전)가 프로그램 실행 → 지정 메뉴 클릭 → 입력을 한 단계씩 가르치면, deskclaw가 그 단계를 매크로(JSON)로 기록한다. 다음부터 [RUN] <name> 은 LLM 없이 결정론적으로 재생한다.

*Same closed loop, two embodiments: skill-cache rules on the manufacturing line, RPA macros on the desktop. Teach once with the LLM; replay forever without it.*

## 5. 보안·거버넌스 / Security & Governance

- **사람이 결정한다 / Human-in-the-loop.** 마스터는 제안만 하고, 중요한 변경은 사람이 승인한다. 승인 = 실행 트리거.
- **keyless 엣지 / Keyless edges.** 엣지에 모델 키가 없으므로 키 유출면이 줄어든다.
- **데이터는 엣지에 / Data stays at the edge.** 원시 telemetry·영상은 공장을 떠나지 않고, 정제된 컨텍스트만 마스터로 간다.
- **감사 / Auditable.** 모든 쓰기·승인은 감사 로그로 남는다(Open MES Korea AuditLog). 결정 동결로 비결정성도 제거된다.
- **최소 인프라 / Minimal footprint.** 브로커리스 HTTP/WS push 우선 — 새 메시지 큐·에이전트 런타임 없이 시작.

## 6. 적용 사례 / Use Case — 분산 엣지 × 제조 MES

dure-factory 데모는 DureClaw 버스와 Open MES Korea(AI-native MES)를 결합한다. 골든 시나리오:

1. **센싱 / Sense** — Pi( executor@pi-cam )·GPU·PC 노드가 키 없이 버스에 합류. 불량 트리거 발생.
2. **종합 / Synthesize** — 마스터가 여러 노드의 신호를 fan-in 수집, 불량 원인을 역할별로 추론·제안.
3. **승인 / Approve** — 품질관리자가 MES UI에서 검토·승인 → quarantine .
4. **물리 회신 / Physical result** — 승인이 버스로 broadcast되면 엣지 노드가 LED·부저·음성으로 발화. AI 결정이 물리 세계로 되돌아온다.
5. **동결 / Freeze** — 승인된 판단은 스킬 캐시 룰로 컴파일, 다음 동일 케이스는 LLM 0회.

*A defect on the line fans out to many machines in ~1 second, Claude synthesizes a structured suggestion, a human approves, the decision returns to the physical world (LED/buzzer), and is frozen into a reusable rule — with an audit trail throughout.*

## 7. 시작점 → AX 여정 / From here to AX

[기존 인프라] → ① 데이터 수집 → ② 분석(LLM) → ③ 자동화(룰 컴파일) → ④ AX  
갈아엮지 않음 collect analyze automate AI transformation

DureClaw는 ①에서 시작해 ④로 가는 *연속적인* 길을 제공한다. 각 단계는 이전 단계의 데이터·승인을 재사용하며, 닫힌 루프가 시간이 갈수록 LLM 의존도를 낮춘다.

## 8. 로드맵 / Roadmap

- **deskclaw** — 접근성 트리 타게팅(좌표→요소, 창 이동에도 견고), 스크린샷→brain 업로드 루프, 매크로 파라미터.
- **edgeclaw** — 더 많은 액추에이터 프로파일(Modbus/OPC-UA 릴레이), 산업 프로토콜 SourceAdapter.
- **버스 / bus** — 결정 동결 룰의 버전 관리·배포, 멀티 워크키 오케스트레이션.
- **MES** — 시계열(EXT-1)·멀티미디어(EXT-2) 종합 조사 고도화, 예지보전(EXT-3).

## 9. 결론 / Conclusion

DureClaw는 *새 AI 플랫폼*이 아니라, **이미 도는 인프라를 AI 크루로 바꾸는 얇은 층**이다. keyless 엣지, 한 두뇌-여러 손, 결정 동결 — 이 세 원칙으로 데이터 수집에서 AX까지 *갈아엮지 않고* 갈 수 있다. 데이터는 엣지 · 두뇌는 분산 · 학습은 닫힌 루프 · 결정은 사람.

*DureClaw is not a new AI platform but a thin layer that turns infrastructure-you-already-run into a collaborating AI crew. Keyless edges, one-brain-many-hands, decision-freezing — three principles to go from data to AX without a rebuild.*

## 부록 A. 와이어 프로토콜 / Wire protocol

| 이벤트 / Event  | 방향         | 의미                                          |
|--------------|------------|---------------------------------------------|
| phx_join     | node → bus | work:<WK> 합류 + presence 등록(역할·capabilities) |
| task.assign  | bus → node | 작업 분배 ( to : 노드명 또는 broadcast )             |
| task.result  | node → bus | 결과 회신 (반드시 연결의 join_ref 포함)                 |
| state.update | bus → node | 상태/결정 전파 (예: decision_<lot>: quarantine )   |
| heartbeat    | node → bus | presence 유지 (15-30s)                        |

## 부록 B. 노드 환경변수 / Node env (공통)

STATE\_SERVER (버스 host:port) · OAH\_SECRET (Bearer) · WORK\_KEY ·  
AGENT\_NAME / AGENT\_ROLE / CAPABILITIES · BRAIN\_URL / BRAIN\_TOKEN (keyless LLM 위임).